Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Сложение K массивов

Группа: М8О-210БВ-25

Студент: Торосян Д.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 16.10.26

Москва, 2026

Постановка задачи

Разработать программу на языке C для поэлементного сложения K массивов одинаковой длины N. Реализовать последовательную и параллельную версии с помощью POSIX Threads. Предусмотреть стратегию, учитывающую количество массивов, их длину и число операций. Максимальное число потоков задавать ключом запуска.

Измерить время обеих версий, исследовать ускорение и эффективность при разных входных данных и числе потоков. Проверить совпадение результатов и показать потоки стандартными средствами операционной системы.

Использованные функции

- pthread_create() и pthread_join() — создание и ожидание потоков.
- pthread_mutex_lock() и pthread_mutex_unlock() — ожидание начала вычислений.
- clock_gettime(CLOCK_MONOTONIC, ...) — измерение длительности вычислений.
- malloc(), free() — память; printf(), fprintf() — вывод текста.
- strtol(), strcmp() — аргументы; exit() — завершение; sleep() — пауза.

Описание алгоритма

Результат — массив R длины N: R[j] = A[0][j] + … + A[K−1][j]. Элементы int хранятся подряд и заполняются по формуле (i % 101) − 50, где i — общий индекс. Суммы имеют тип long long.

Последовательная версия складывает все массивы. Работа оценивается как W = K · N. Выбирается p = min(лимит, max(1, W / 10000)), деление целочисленное. Порог 10000 оценивает полезную работу на поток. При p = 1 новые потоки не создаются; иначе создаются p−1 потоков, а основной выполняет ещё одну часть работы.

Если N / p < 16 и K ≥ p, массивы делятся между потоками. Каждый поток получает свои массивы и считает отдельную частичную сумму. После pthread_join основной поток складывает эти результаты. Между частичными суммами оставляется 128 байт, чтобы уменьшить взаимное влияние потоков через кэш.

Иначе поток i обрабатывает индексы результата от N · i / p до N · (i + 1) / p, не включая правую границу. Он читает все массивы, но записывает только свой участок. Блокировки при сложении не нужны: мьютекс используется только перед началом работы.

Метрики производительности

Ускорение S показывает, во сколько раз параллельная версия быстрее последовательной. Tseq — время последовательной версии, Tp — время параллельной версии, p — фактически использованное число потоков, включая основной.

$$S = \frac{Tseq}{Tp}$$

Эффективность E — отношение ускорения к числу потоков в процентах. Она показывает отдачу от распараллеливания и не равна загрузке процессора.

$$E = \frac{S}{p} \cdot 100\,\%$$

Код программы

main.c

```
#define _POSIX_C_SOURCE 200809L
#include "arrays.h"


#include <errno.h>
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>


void error(const char *text) {
    fprintf(stderr, "%s", text);
    exit(1);
}


int read_number(const char *text) {
    char *end;
    errno = 0;
    long value = strtol(text, &end, 10);
    if (errno != 0 || *end != '\0' || value <= 0 || value > INT_MAX) {
        error("Ожидалось положительное целое число.\n");
    }
    return (int)value;
}


double current_time_ms(void) {
    struct timespec time;
    if (clock_gettime(CLOCK_MONOTONIC, &time) != 0) {
        error("Ошибка измерения времени.\n");
    }
    return time.tv_sec * 1000.0 + time.tv_nsec / 1000000.0;
}


int main(int argc, char *argv[]) {
    if ((argc != 7 && argc != 8) || strcmp(argv[1], "-k") != 0 ||
        strcmp(argv[3], "-n") != 0 || strcmp(argv[5], "-t") != 0 ||
        (argc == 8 && strcmp(argv[7], "-s") != 0)) {
        error("Использование: ./main -k K -n N -t ПОТОКИ [-s]\n");
    }
    int k = read_number(argv[2]);
    int n = read_number(argv[4]);
    int limit = read_number(argv[6]);
    int inspect = argc == 8;
    if (limit > 1024) error("Допустимо не больше 1024 потоков.\n");


    Arrays arrays;
    arrays_create(&arrays, (size_t)k, (size_t)n);
    arrays_fill(&arrays);
    long long *sequential = allocate((size_t)n * sizeof(long long));
    long long *parallel = allocate((size_t)n * sizeof(long long));
    int threads = choose_threads(&arrays, limit);


    // Прогрев памяти перед замерами.
    sum_sequential(&arrays, sequential);
    sum_sequential(&arrays, parallel);
    double start = current_time_ms();
    sum_sequential(&arrays, sequential);
    double sequential_ms = current_time_ms() - start;
    start = current_time_ms();
    sum_parallel(&arrays, parallel, threads, inspect);
    double parallel_ms = current_time_ms() - start;
```

```
    int ok = results_equal(sequential, parallel, (size_t)n);

    printf("Массивов: %d\nДлина: %d\n", k, n);
    printf("Лимит потоков: %d\nИспользовано потоков: %d\n", limit, threads);
    printf("Стратегия: %s\n", threads == 1 ? "последовательная" :
           (n / threads < 16 && k >= threads ? "по массивам" : "по элементам"));
    if (!inspect) {
        double speedup = parallel_ms > 0 ? sequential_ms / parallel_ms : 0;
        printf("Последовательно, мс: %.3f\n", sequential_ms);
        printf("Параллельно, мс: %.3f\n", parallel_ms);
        printf("Ускорение: %.3f\n", speedup);
        printf("Эффективность, %%: %.3f\n", speedup / threads * 100);
    }
    printf("Проверка: %s\n", ok ? "OK" : "ОШИБКА");
    if (n <= 20) {
        printf("Сумма:");
        for (int j = 0; j < n; j++) printf(" %lld", parallel[j]);
        printf("\n");
    }
    arrays_free(&arrays);
    free(sequential);
    free(parallel);
    return ok ? 0 : 1;
}
```

arrays.h

```
#ifndef ARRAYS_H
#define ARRAYS_H

#include <stddef.h>

typedef struct {
    size_t count;
    size_t length;
    int *data;
} Arrays;

void error(const char *text);
void *allocate(size_t bytes);
void arrays_create(Arrays *arrays, size_t count, size_t length);
void arrays_free(Arrays *arrays);
void arrays_fill(Arrays *arrays);
void sum_part(const Arrays *arrays, long long *result,
              size_t first_array, size_t last_array, size_t begin, size_t end);
void sum_sequential(const Arrays *arrays, long long *result);
int choose_threads(const Arrays *arrays, int limit);
void sum_parallel(const Arrays *arrays, long long *result, int threads, int inspect);
int results_equal(const long long *a, const long long *b, size_t length);

#endif
```

arrays.c

```
#include "arrays.h"

#include <stdint.h>
#include <stdlib.h>

void *allocate(size_t bytes) {
    void *memory = malloc(bytes);
    if (memory == NULL) error("Не удалось выделить память.\n");
    return memory;
}
```

```
void arrays_create(Arrays *arrays, size_t count, size_t length) {
    if (count > SIZE_MAX / sizeof(int) / length ||
        length > SIZE_MAX / sizeof(long long)) {
        error("Слишком большой размер массивов.\n");
    }
    arrays->count = count;
    arrays->length = length;
    arrays->data = allocate(count * length * sizeof(int));
}


void arrays_free(Arrays *arrays) {
    free(arrays->data);
}


void arrays_fill(Arrays *arrays) {
    for (size_t i = 0; i < arrays->count * arrays->length; i++) {
        arrays->data[i] = (int)(i % 101) - 50;
    }
}


void sum_part(const Arrays *arrays, long long *result,
              size_t first_array, size_t last_array, size_t begin, size_t end) {
    for (size_t j = begin; j < end; j++) result[j] = 0;
    for (size_t i = first_array; i < last_array; i++) {
        for (size_t j = begin; j < end; j++) {
            result[j] += arrays->data[i * arrays->length + j];
        }
    }
}


void sum_sequential(const Arrays *arrays, long long *result) {
    sum_part(arrays, result, 0, arrays->count, 0, arrays->length);
}


int results_equal(const long long *a, const long long *b, size_t length) {
    for (size_t i = 0; i < length; i++) {
        if (a[i] != b[i]) return 0;
    }
    return 1;
}
```

parallel.c

```
#define _POSIX_C_SOURCE 200809L
#include "arrays.h"


#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>


typedef struct {
    const Arrays *arrays;
    long long *result;
    size_t first_array, last_array;
    size_t begin, end;
    pthread_mutex_t *start;
} ThreadData;


void *worker(void *argument) {
    ThreadData *data = argument;
    pthread_mutex_lock(data->start);
```

```
    pthread_mutex_unlock(data->start);
    sum_part(data->arrays, data->result, data->first_array, data->last_array,
             data->begin, data->end);
    return NULL;
}


int choose_threads(const Arrays *arrays, int limit) {
    size_t useful = arrays->count * arrays->length / 10000;
    if (useful == 0) useful = 1;
    return useful < (size_t)limit ? (int)useful : limit;
}


void sum_parallel(const Arrays *arrays, long long *result, int threads, int inspect) {
    if (threads == 1 && !inspect) {
        sum_sequential(arrays, result);
        return;
    }
    int split_arrays = arrays->length / threads < 16 &&
                       arrays->count >= (size_t)threads;
    size_t length = arrays->length;
    // Между частичными суммами оставляем 128 байт для кэша.
    size_t stride = length + 16;
    long long *partial = NULL;
    if (split_arrays) partial = allocate((size_t)threads * stride * sizeof(long long));
    pthread_t ids[threads];
    ThreadData data[threads];
    pthread_mutex_t start = PTHREAD_MUTEX_INITIALIZER;
    pthread_mutex_lock(&start);


    for (int i = 0; i < threads; i++) {
        data[i].arrays = arrays;
        data[i].result = split_arrays ? partial + (size_t)i * stride : result;
        data[i].first_array = split_arrays ? arrays->count * (size_t)i / threads : 0;
        data[i].last_array = split_arrays ?
                             arrays->count * (size_t)(i + 1) / threads : arrays->count;
        data[i].begin = split_arrays ? 0 : length * (size_t)i / threads;
        data[i].end = split_arrays ? length : length * (size_t)(i + 1) / threads;
        data[i].start = &start;
        if (i > 0 && pthread_create(&ids[i], NULL, worker, &data[i]) != 0) {
            error("Не удалось создать потоки. Уменьшите -t.\n");
        }
    }
    if (inspect) {
        printf("Потоки созданы. Пауза 6 секунд для просмотра в ОС.\n");
        fflush(stdout);
        sleep(6);
    }
    pthread_mutex_unlock(&start);
    worker(&data[0]);
    for (int i = 1; i < threads; i++) {
        if (pthread_join(ids[i], NULL) != 0) error("Ошибка ожидания потока.\n");
    }
    if (split_arrays) {
        for (size_t j = 0; j < length; j++) {
            result[j] = 0;
            for (int i = 0; i < threads; i++) result[j] += partial[(size_t)i * stride + j];
        }
        free(partial);
    }
    pthread_mutex_destroy(&start);
}
```

Протокол работы программы

Сборка программы и запуск для 64 массивов длины 1000000 с лимитом 14 потоков:

```
make
./main -k 64 -n 1000000 -t 14
```

Пример результата одного из выполненных запусков:

```
Массивов: 64
Длина: 1000000
Лимит потоков: 14
Использовано потоков: 14
Стратегия: по элементам
Последовательно, мс: 8.146
Параллельно, мс: 2.387
Ускорение: 3.413
Эффективность, %: 24.376
Проверка: OK
```

Ключ -k задаёт число массивов, -n — их длину, -t — предел потоков от 1 до 1024. При N ≤ 20 программа также выводит сумму. Например, для ./main -k 3 -n 5 -t 4 результат равен −135 −132 −129 −126 −123; используется один поток, поскольку данных мало.

Просмотр потоков средствами ОС

На macOS команда make stats THREADS=14 запускает программу с ключом -s, который удерживает созданные потоки шесть секунд. В это время Makefile выполняет ps -M -p PID. В проверочном запуске для PID 46888 получено 14 строк потоков: основной и 13 созданных через pthread_create. Время в режиме -s не выводится, поскольку в него попала бы пауза.

```
make stats THREADS=14
```

На Linux просмотр выполняется командой ps -L. Пример запуска в том же терминале:

```
./main -k 64 -n 100000 -t 14 -s &
pid=$!; sleep 2; ps -L -p "$pid"; wait "$pid"
```

Результаты экспериментов

Эксперименты выполнены 16.10.2026 на Apple M4 Pro с 14 логическими ядрами и 48 ГиБ оперативной памяти, macOS 26.6.2. Программа собрана Apple Clang с оптимизацией -O2. Для каждой конфигурации выполнено три запуска, в таблицах приведены средние значения.

Перед замерами прогреваются оба буфера результата. Генерация, прогрев и проверка не учитываются. В Tp входят создание потоков, работа со служебной памятью, вычисление, ожидание и объединение сумм. База Tseq — среднее трёх последовательных измерений при -t 1, общее для строк одного набора.

Набор 1 — K = 64, N = 1000000

Число сложений W = 64000000. Tseq = 8,200 мс. При p > 1 работа делится по элементам.

Число потоков	Время исполнения, мс	Ускорение	Эффективность, %
1	8,370	0,980	97,97
2	4,427	1,852	92,62
4	2,910	2,818	70,46
8	2,274	3,606	45,08
14	2,681	3,059	21,85
64	2,531	3,240	5,06
128	2,916	2,812	2,20
1024	13,092	0,626	0,06

Набор 2 — K = 8000000, N = 8

Число сложений W = 64000000. Tseq = 11,128 мс. При p > 1 работа делится по массивам.

Число потоков	Время исполнения, мс	Ускорение	Эффективность, %
1	10,952	1,016	101,61
2	4,982	2,234	111,68
4	2,765	4,025	100,62
8	2,710	4,107	51,34
14	2,509	4,435	31,68
64	2,461	4,522	7,07
128	2,899	3,839	3,00
1024	12,068	0,922	0,09

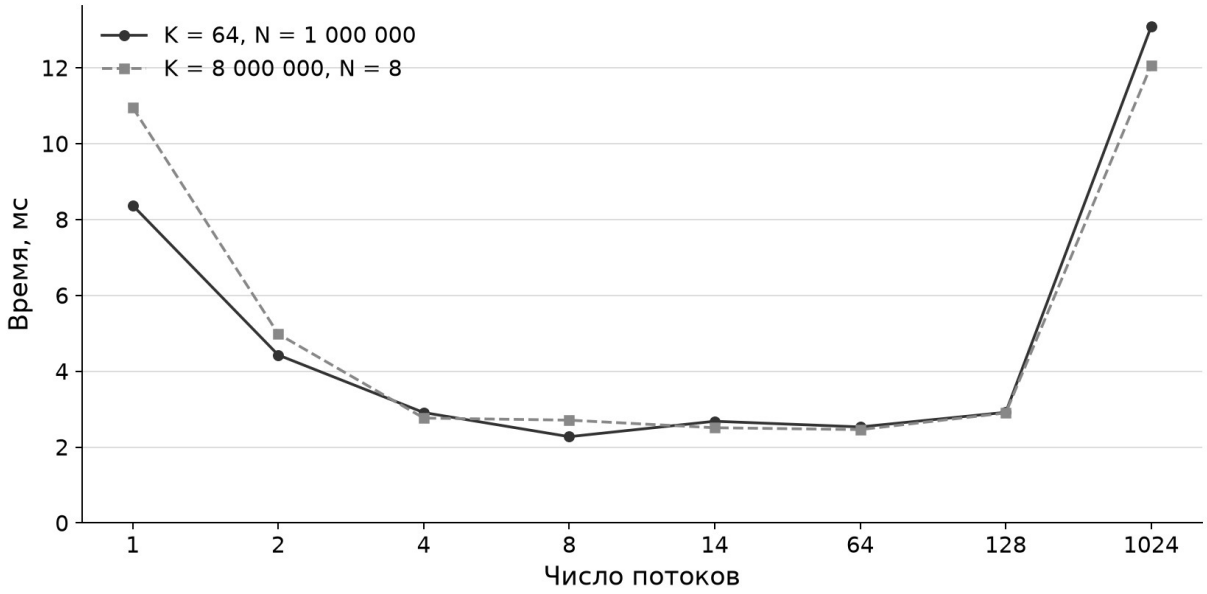


Рисунок 1 — Зависимость времени выполнения от числа потоков

Влияние объёма данных

При K = 64 и лимите 14 потоков получены следующие результаты. Каждый размер также проверен тремя запусками; S вычислено по средним Tseq и Tp для этой строки.

Длина N	Потоки p	Tseq, мс	Tp, мс	Ускорение	E, %
2048	13	0,012	0,151	0,080	0,61
16384	14	0,132	0,192	0,688	4,91
1000000	14	8,388	2,681	3,129	22,35

Для K = 64 и N = 128 объём работы равен 8192 операциям, поэтому адаптация оставляет один поток. Численные метрики для этого размера не приведены: время близко к точности вывода программы 0,001 мс.

Анализ результатов

Для длинных массивов минимальное среднее время 2,274 мс получено при 8 потоках: ускорение 3,606 и эффективность 45,08 %. Для большого числа коротких массивов лучший результат также получен при 64 потоках: 2,461 мс, ускорение 4,522 и эффективность 7,07 %. Это лучшие значения в данной серии из трёх запусков для каждой конфигурации.

Для коротких массивов времена при 14 и 64 потоках отличаются примерно на 2 %, хотя эффективность при 64 потоках значительно ниже. Такую разницу нельзя считать устойчивым преимуществом по трём запускам. При 128 и особенно 1024 потоках время растёт: число ядер остаётся прежним, а расходы на создание, переключение и ожидание потоков увеличиваются. Скорость сложения также ограничивается пропускной способностью памяти.

Одинаковое количество сложений не означает одинаковое время: при разных K и N меняются длина внутренних циклов, работа с кэшем и способ разделения данных. Для малого объёма данных создание потоков обходится дороже самого сложения; это видно по

S < 1 в таблице объёмов. Порог 10000 операций является приближённым и не гарантирует ускорения.

Измерения зависят от планировщика, кэша, частоты процессора и фоновой нагрузки, поэтому время меняется немонотонно. В первом наборе восемь потоков оказались быстрее четырнадцати. Во втором наборе при двух и четырёх потоках расчётная эффективность превысила 100 %. Это может быть связано с изменением работы памяти и разбросом времени относительно выбранной базы Tseq; E не является процентом загрузки процессора. Для уточнения таких результатов нужны более длинные серии измерений.

Вывод

Реализованы последовательное и параллельное сложение K массивов на языке C. Стратегия выбирается по числу операций, количеству массивов и их длине. Использованы POSIX Threads, мьютекс для начала работы и pthread_join для ожидания. Ограничение -t учитывает основной поток; его выполнение подтверждено средствами ОС. Результаты обеих версий совпали во всех выполненных запусках.

В этой серии лучшее время для длинных массивов получено при 8 потоках, для коротких — при 64, причём результаты 14 и 64 потоков близки. Число ядер само по себе не определяет лучший параметр. При избыточном числе потоков растут накладные расходы, а на малых данных выгоднее последовательная обработка.